ARM PrimeCell

Static Memory Controller (PL092)

Integration Manual

ARM

ARM PrimeCell Static Memory Controller (PL092) Integration Manual

Proprietary notice

Document confidentiality status

This document is Open Access (no restriction on distribution).

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell Static Memory Controller

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A00	23rd April, 2001	First draft
A01	19th May,2003	Updated for r1p3 release

1.2 References

This document refers to the following documents.

Ref.	Doc. No.	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0203A	ARM PrimeCell Static Memory Controller (PL092) Technical Reference Manual
3	PL092 DDES 0000	ARM PrimeCell Static Memory Controller (PL092) Design Manual

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
RTL	Register Transfer Level
SMC	Static Memory Controller
STA	Static Timing Analysis

2 SCOPE

This document describes how the ARM PrimeCell Static Memory Controller (SMC PL092) may be integrated into an AMBA-based ASIC when using a typical industry standard ASIC design flow.

3 INTRODUCTION

The ARM PrimeCell Static Memory Controller is a reusable soft-IP block that has been developed with the prime aim of reducing time to market for ASIC development.

A carefully chosen set of design rules have been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customised to suit specific customer requirements.

For further information on AMBA, refer to the AMBA Specification [Ref.1].

For detailed technical information and programming data on the ARM PrimeCell block, refer to the ARM PrimeCell Static Memory Controller (PL092) Technical Reference Manual [Ref.2].

The ARM PrimeCell Static Memory Controller (PL092) Design Manual [Ref.3] contains information about the implementation and design of the ARM PrimeCell Static Memory Controller block that may be needed for optional customisation of this block.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA.

The AMBA Interconnection schemes, Application Note 05 describes on-chip bus interconnection using OR bus, multiplexer and tri-state within an AMBA system [Ref.6].

4 INTEGRATION

4.1 Design Flow Supported

The ARM PrimeCell Static Memory Controller has been designed to allow integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimisation of area (gate count) and power-consumption, ease of integration has been given higher priority to aid design reuse.

The following sections of this document address issues that may arise during ASIC system-level integration of the ARM PrimeCell Static Memory Controller block.

An example ASIC design flow is described below:

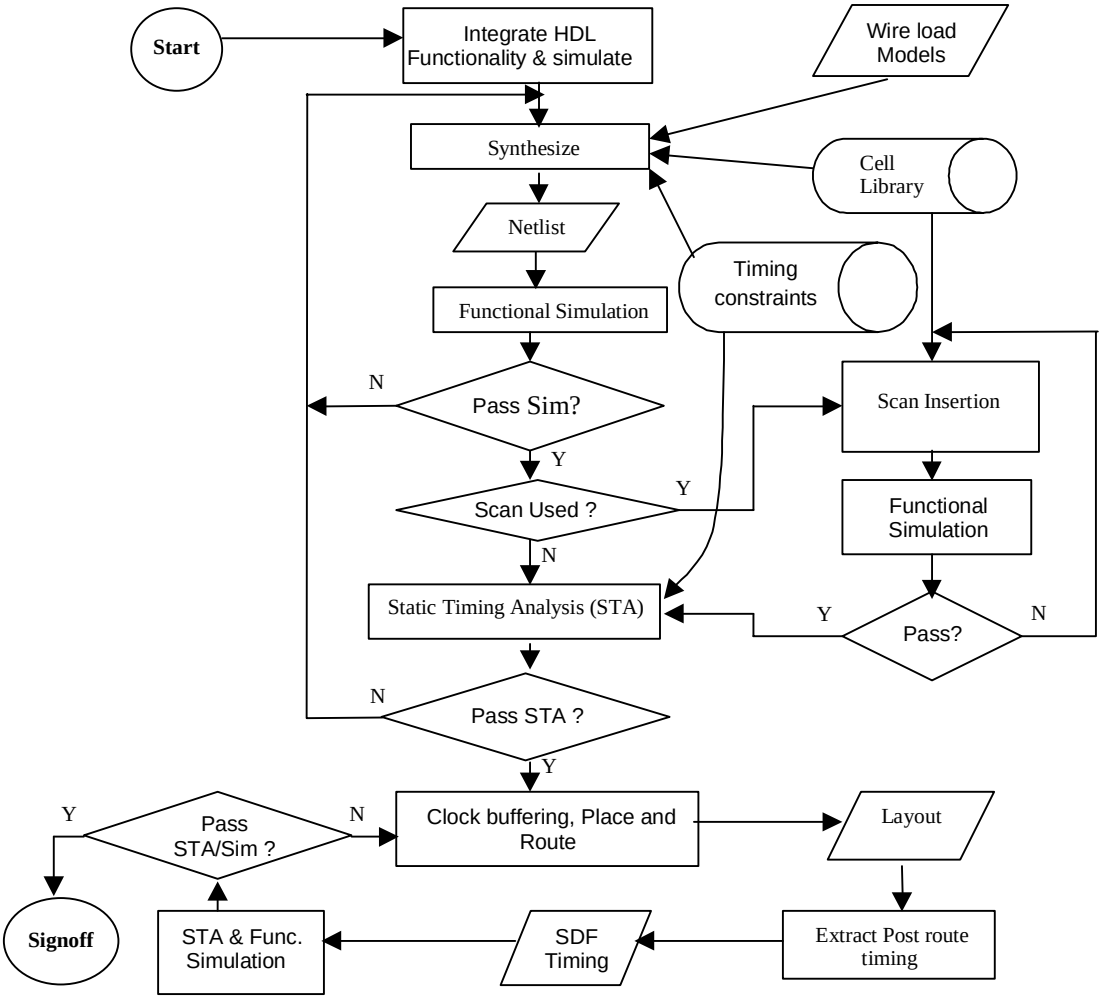

Figure 4.1-1: Example ASIC Design Flow supported

As an alternative to dynamic functional simulation, it is also possible to use logic equivalence checking.

4.2 Signal Connectivity

The following tables describe the I/O pins for the ARM PrimeCell Static Memory Controller block. For further information refer to the ARM PrimeCell Static Memory Controller (PL092) Technical Reference Manual [Ref.2].

Signal Name	Type (Input/ Output)	Source / Destination	Description
nHCLK	Input	Clock source	Inverted Bus Clock. This clock is used for the generation of the write enable output.
HCLK	Input	Clock source	Bus Clock. This clock times all bus transfers. All signal timings are related to the rising edge.
HRESETn	Input	Reset Controller	Bus reset, active low, used to reset the system and bus.
HREADYIN	Input	Other AHB Slaves	Transfer completed input. When HIGH, indicates that a transfer has finished on the bus. This signal may be driven LOW to extend a transfer.

***Table* 4.1. Common AMBA AHB signal descriptions (for AHB master and slave)**

Signal Name	Type (Input/ Output)	Source / Destination	Description
HADDR[28:0]	Input	AHB Master	Address bus. Subset of the 32-bit system address bus.
HTRANS[1:0]	Input	AHB Master	Transfer type, indicates the type of current transfer: NONSEQUENTIAL, SEQUENTIAL, IDLE or BUSY.
HWRITE	Input	AHB Master	Transfer direction. When HIGH, this signal indicates a write transfer to this block, and when LOW a read from this block.
HSIZE[2:0]	Input	AHB Master	Transfer size, indicates the size of the current transfer, typically byte (8-bit), halfword (16-bit) or word (32-bit).
HBURST[2:0]	Input	AHB Master	Indicates if the transfer forms part of a burst. Four, eight and sixteen beat bursts are supported and the burst may be either incrementing or wrapping.
HWDATA[31:0]	Input	AHB Master	Write data bus, used to transfer data to this block.
HSELSMC	Input	AHB Decoder	Slave select signal to access memory banks through SMC.
HSELREG	Input	AHB Decoder	Slave select to access the control and status registers of SMC.
HRDATA[31:0]	Output	AHB Slave	Read data bus, used to read data from this block.
HREADYOUT	Output	AHB Slave	Transfer done output. When HIGH, indicates that a transfer has finished on the bus. This signal may be driven LOW to extend a transfer.
HRESP[1:0]	Output	AHB Slave	Transfer response, The transfer response provides additional information on the status of a transfer. Two different responses are possible, OKAY and ERROR.

Table 4.1-1 AHB slave signals related to SmcCore block

Signal Name	Type (Input/ Output)	Source / Destination	Description
HRESPTIC[1:0]	Input	AHB Slave	Transfer response, The transfer response provides additional information on the status of a transfer. The TIC supports both SPLIT and RETRY responses.
HRDATATIC[31:0]	Input	AHB Slave	The read data bus is used to transfer data from bus slaves to the bus master during test read operations.
HGRANTTIC	Input	AHB Arbiter	This signal indicates that the TIC is currently the highest priority master. Ownership of the address/control signals changes at the end of the transfer when **HREADYIN** is HIGH.
HADDRTIC[31:0]	Output	AHB Slave	The 32-bit system address bus.
HTRANSTIC[1:0]	Output	AHB Slave	Indicates the type of the current transfer, which can be Non-Sequential, Sequential or Idle. The TIC does not use the Busy transfer type.
HWRITETIC	Output	AHB Slave	When HIGH this signal indicates a write transfer and when LOW a read transfer.
HSIZETIC[2:0]	Output	AHB Slave	Indicates the size of the transfer, which is typically byte (8-bit), halfword (16-bit) or word (32-bit). The TIC does not support larger transfer sizes.
HBURSTTIC[2:0]	Output	AHB Slave	Indicates if the transfer forms part of a burst. The TIC always performs incrementing bursts of unspecified length.
HPROTTIC[3:0]	Output	AHB Slave	The Protection control signals indicate if the transfer is an op-code fetch or data access, as well as if the transfer is a supervisor mode access or user mode access. These signals can also indicate whether the current access is cacheable or bufferable.
HWDATATIC[31:0]	Output	AHB Slave	The write data bus is used to transfer data from the master to bus slaves during write operations. A minimum data bus width of 32 bits is recommended, however this may easily be extended to allow for higher bandwidth operation.
HBUSREQTIC	Output	AHB Arbiter	A signal from the TIC to the bus arbiter, which indicates that it requires the bus.
HLOCKTIC	Output	AHB Arbiter	When HIGH this signal indicates that the TIC requires locked access to the bus and no other master should be granted the bus until this signal is LOW.

Table 4.1.2 AHB Master signals related to TIC block

Signal Name	Type (Input/ Output)	Source / Destination	Description

BIGENDIAN	Input	System	This static configuration bit indicates the type of endianess of the memory system: 0 = little-endian 1 = big-endian
REMAP	Input	System	Indicates the state of the memory map: 0 = reset memory map (SMCS7 mapped to SMCS0) 1 = normal memory map
MCBUSREQ	Input	Additional Memory Controller	Bus request signal from the Additional Memory Controller block in the system. This is an input to the internal DBI module inside the SMC.
MCADDR[25:0]	Input	Additional Memory Controller	Address bus from the Additional Memory Controller block to be routed to the memory address bus, after arbitration by DBI module inside the SMC.
MCDATAOUT[31:0]	Input	Additional Memory Controller	Output data bus lines from the Additional Memory Controller block to be routed to the memory data bus, after arbitration by DBI module inside the SMC.
MCDATAEN[3:0]	Input	Additional Memory Controller	Output pad enables from the Additional Memory Controller block. These will be routed to the final pad enables, SMDATAEN lines after arbitration by the DBI.
TICBUSGNTEBI	Input	EbiSdram	Bus grant input to TIC from an external EbiSdram module
SMBUSGNTEBI	Input	EbiSdram	Bus grant input to SmcCore block from an external EbiSdram
MCBUSGNT	Output	Additional Memory Controller	Bus grant signal from the DBI to the Additional Memory Controller. The Additional Memory Controller can then take control of the external address and data bus.
TICBUSREQEBI	Output	EbiSdram	External memory data bus request signal from the TIC to the external EbiSdram block
SMBUSREQEBI	Output	EbiSdram	External memory data bus request signal from the SmcCore to the external EbiSdram block
TICREADEBI	Output	EbiSdram	Pad Enable signal from TIC when EbiSdram is to be used for bus arbitration
TBUSOUTEBI	Output	EbiSdram	Data bus output from the TIC when EbiSdram is to be used for bus arbitration

***Table* 4.1. Internal signals**

Signal Name	Type (Input/ Output)	Source / Destination	Description
SCANENABLE	Input	Test Controller	Scan Enable signal. Indicates that the circuit is in scan-test mode.
SCANINHCLK	Input	Test Controller	Scan data input of HCLK scan chain, for clocking in the test pattern in scan-test mode.
SCANINnHCLK	Input	Test Controller	Scan data input of nHCLK scan chain, for clocking in the test pattern in scan-test mode.
SCANOUTHCLK	Output	Test Controller	Scan data output of HCLK scan chain, for observing the flip-flop contents in scan test mode.
SCANOUTnHCLK	Output	Test Controller	Scan data output of nHCLK scan chain, for observing the

			flip-flop contents in scan test mode.

Table 4.1.6 Scan Test related signals

Signal Name	Type (Input/ Output)	Source / Destination	Description
SMMWCS7[1:0]	Input	Input Pad	These static configuration bits indicate the memory width used for boot memory bank one: 00 = 8-bit 01 = 16-bit 10 = 32-bit 11 = No Change
SMRBLECS7	Input	Input Pad	This static configuration bit is used to configure the RBLE pin immediately after reset.
SMWAIT	Input	Input Pad	Wait mode input from external memory controller. Active HIGH or active LOW (default) as programmed in the SMC control registers for each bank.
CANCELSMWAIT	Input	Input Pad	Allow the system to recover from an externally waited transfer that takes longer than expected to finish. Active HIGH.
SMDATAIN[31:0]	Input	Input Pad	External input data bus used to read data from memory bank.
SMDATAOUT[31:0]	Output	Output Pad	External output data used to write data from SMC to memory bank.
SMADDR[25:0]	Output	Output Pad	External memory address bus, to external memory banks.
SMCS[7:0]	Output	Output Pad	Chip selects for bank-0 to bank-7 of external memory, active LOW by default.
nSMDATAEN[3:0]	Output	Output Pad	Tri-state I/O pad enable for the byte lanes of the external memory data bus **SMDATA[31:0]**, active LOW. Enables the byte lanes [31:24], [23:16], [15:8], [7:0] of the data bus independently.
nSMWEN	Output	Output Pad	Write enable for the external memory banks, active LOW.
nSMBLS[3:0]	Output	Output Pad	Byte lane select signals, active LOW. The signals **nSMBLS[3:0]** select byte lanes [31:24], [23:16], [15:8], [7:0] on the data bus.
nSMOEN	Output	Output Pad	Output enable for external memory banks, active LOW.
TESTREQA	Input	Input Pad	This is the Test Bus Request ‘A’ input signal and is required as a dedicated device pin. During normal system operation the **TESTREQA** signal is used to request entry into the test mode. During test **TESTREQA** is used, in combination with **TESTREQB**, to indicate the type of test vector that will be applied in the following cycle.
TESTREQB	Input	Input Pad	During test this signal is used, in combination with

			TESTREQA, to indicate the type of test vector that will be applied in the following cycle.
TESTACK	Output	Output Pad	The test bus acknowledge signal gives external indication that the TIC has been granted and also indicates when a test access has completed. When **TESTACK** is LOW the current test vector must be extended until **TESTACK** becomes HIGH.
EXTBUSMUX	Input	Input Pad	This pin should be tied to logic LOW or logic HIGH to choose between the internal DBI inside SMC or an external EbiSdram of the system, respectively [depending on which of the two will be used for bus arbitration]

Table 4. Signals to I/O pad cells

4.3 Bus Interconnection

The Static Memory Controller block interfaces with the AMBA AHB interface using separate uni-directional read data bus, HRDATA and write data bus, HWDATA. The HRDATA is an output bus in case of the SmcCore block (which is an AHB slave) and input bus in the case of TIC (which is an AHB master). The HWDATA is an input in case of the SmcCore block (AHB slave) and output bus in the case of TIC (AHB master). For further information refer to the AHB Specification [Ref.1].

The data bus is interfaced to the Bus Masters by using multiplexers with the data buses from the other AHB slave peripheral blocks as inputs of the multiplexer. This is called the 'MUXBUS' implementation. The HREADYOUT is driven HIGH by default.

However, it is also possible to interface the Static Memory Controller block with a 3-state bi-directional data bus. A wrapper needs to be added around the Static Memory Controller block connecting the read and write data buses to bi-directional drivers. Control of the 3-state drivers can be done using the AHB control signals. A bi-directional 3-state implementation, while yielding a small area saving, will make system-level static timing analysis more difficult, as well as causing problems in scan test/ATPG and in achieving high coverage fault grading.

4.4 Endianness

The Static Memory Controller has an input pin called **BIGENDIAN**. This input is used in the SmcCore, which is designed to be configured for either Big-Endian or Little-Endian memory organisation during the system integration stage. The BIGENDIAN top-level port should be connected HIGH for Big-Endian operation or connected LOW for Little-Endian. The SmcCore does not support dynamic Endianness. This means that, once this input is held at a particular value [indicating a particular Endianness], then the whole system is assumed to be configured for that Endianness.

The state of the BIGENDIAN port provides information to the SmcCore block so that half word and byte data are generated/routed on the appropriate byte lanes of the data bus. Note that irrespective of Endianness, the memory device connections to the external data bus (SMDATA) should be made starting from the least significant bit. For example, an 8-bit memory device should be connected to SMDATA[7:0] in both Little-Endian and Big-Endian modes. Similarly a 16-bit memory device should be connected to SMDATA[15:0] and so on.

During memory read accesses, for transfer sizes less than 32-bits, the data will be routed only to the appropriate byte lanes. The other byte lanes are not updated. It is the master's responsibility to get data from the correct byte lanes depending on the Endianness. Register accesses to the SmcCore block, in the SMC, must be word only since data is not driven on unused portions of the data bus for half word or byte accesses to the SMC registers.

4.5 Customisation

The ARM PrimeCell Static Memory Controller can be modified to suit specific user configurations, but the full benefit of a fast integration will be only obtained when the design is used without modification. In the event of modification, it is the user's responsibility to ensure that the design is fully functional by appropriate amendment of the functional test program and the testbenches where required. Information to support such modification is included in the Static Memory Controller (SMC_PL092) Design Manual [Ref.4].

4.5.1 Automatic boot memory width configuration at Reset

The SMC allows the size of bank seven (selected with **SMCS7** chip select line) to be configured at reset using the external control pins **SMMWCS7[1:0]**. This feature is used to allow the boot memory size to be chosen at reset [the external boot ROM should be connected to Bank-7]. This ensures that the Bank-7 can always have the correct memory width setting for the respective boot memory device. After reset, if the REMAP input pin is held LOW, then the Bank-7 is mapped on to Bank-0 for boot purposes.

The SMC also allows the RBLE of bank seven (selected with SMCS7 chip select lane) to be configured at reset using external control pin **SMRBLECS7**. This feature allows reading from 16 bit or 32 bit devices after reset.

For more related information please refer to ARM PrimeCell Static Memory Controller Technical Reference Manual [2].

4.5.2 Selection between the in-built DBI or an external EBI for the memory bus arbitration

Option has been provided in the SMC peripheral so that the user can choose between the internal DBI block or the external EBI, like EbiSdram. These bus arbiters are used for interfacing and sharing the external memory address and data pins. The user needs to tie the **EXTBUSMUX** input pin to either logic HIGH or logic LOW. If the EbiSdram is to be used then the **EXTBUSMUX** input pin should be tied to logic HIGH. The internal DBI block, of the SMC, then becomes redundant. For such cases, a bypass logic has been implemented so that it is possible to optimise out the DBI block during synthesis. The following input-output pins of the SMC will then become active : **TICBUSREQEBI, SMBUSREQEBI, TICBUSGNTEBI, SMBUSGNTEBI, TICREADEBI** and **TBUSOUTEBI**. In addition the Additional Memory Controller's I/O's [viz. **MCBUSREQ, MCBUSGNT, MCADDR, MCDATAOUT and MCDATAEN**] will become inactive and should not be used.

When the internal DBI block is used for bus interface, then the **TICBUSREQEBI, SMBUSREQEBI, TICBUSGNTEBI, SMBUSGNTEBI, TICREADEBI** and **TBUSOUTEBI** I/O pins of the SMC become inactive, so they should not be used.

4.6 Synthesis

The Static Memory Controller block has been synthesised using the Synopsys Design Compiler synthesis tool with the scripts located in the synopsys directory. The version 2000.11 of the Design Compiler has been used.

A README text file in the synopsys directory provides details of the synthesis directory structure used.

The following synthesis strategy has been used for the SMC : The SMC top level is made up of three main blocks, which are SmcCore, TIC and the DBI. These blocks are separately synthesised and then linked together at the top level to get the final structural SMC netlist. Out of the three, the SmcCore and the DBI blocks are targeted at 100 MHz. The TIC, however, will never work at this frequency and so is targeted for compile at 10 MHz. During the top level compile the TIC input to output paths are set as false paths, to ensure that synopsys does not attempt to optimise the TIC related paths.

The master compile scripts exists for each block, which are as follows –

- For SMC top level called Smc.cmd
- The SmcCore block, named as SmcCore.cmd

- DBI block, named as DBI.cmd
- TIC block, named as TIC.cmd

The native synopsys ‘db’ file format output, as a result of the lower level SmcCore, DBI and TIC blocks synthesis compile, are linked together and the SMC is synthesised to the chosen target cell library. For the compiles without scan insertion, the lower level blocks are synthesised normally. The scan insertion is done only at the top level SMC compile. If scan-insertion is to be performed the master compile script of the SMC invokes the Smc_scan.cmd script. For the scan-insertion compile at the SMC top level, the lower blocks are synthesised with the “scanready” test methodology {Please refer to **Appendix A**}. Here the normal flip-flops are just replaced by the equivalent multiplexed scan flip-flops without any scan chain formation. The resultant ‘dbs’ are linked together at the top level and then the scan chain is formed.

The final area and gate count that will be achieved is dependent on the target cell library used, and synthesis timing constraints and command options.

The synthesis scripts consist of the following three main categories:

4.6.1 Setup Scripts

Scripts that are common to several designs are located in the $GLOBAL/synopsys_shared/scr_common directory. [Please refer to **Appendix A** for the list of PrimeCell world environment variables]. The following files need several parameters or variables to be changed in order to target a different cell library.

The setup script, ‘setup.scr’ defines the following:

- Synopsys variable settings
- UNIX directory path for Synopsys read and write caches

The ‘global.scr’ script may need modifying to define the following :

- target area goals
- maximum transition costs

The ‘library_avanti_cb25_v2.1.scr’ script is included by the master compile script to specify the target library and the cells in the target library that should not be used. It is recommended that an equivalent file be created for the target library as per library vendor or customer requirements. It is also a practice to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. The user must ensure that the ‘library_avanti_cb25_v2.1.scr’ file is modified and renamed appropriately for the chosen technology library and included by the master compile script.

The ‘$LIB_SYNOP’ directory must contain the files, or links to the target cell library files, in Synopsys format.

4.6.2 Command Script

The master compile command scripts located in the synopsys/scripts directory, analyses the corresponding HDL source before mapping and optimising to a specific target technology cell library. These scripts include setup and constraint scripts to set timing and design rule constraints on the design, prior to optimisation. The compiled design is written out both in native Synopsys .db format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDF files and various reports for timing and violations are also written out. Additionally the run-time log is redirected into a separate log/ directory.

The scan-insertion script Smc_scan.cmd in the synopsys/scripts directory performs scan insertion for the design.

Prior to synthesis, several variables need to be set. The scripts use environment variables, which are assigned to synthesis variables in the scripts. The environment variables provide flexibility without requiring editing of the scripts. The following environment variables are used:

- $PERIPH is assigned to ‘topname’ to define the name of the top level module in the design.

- $HDL_SOURCE is assigned to ‘hdl’ to define the input HDL source. HDL_SOURCE should be set to either vhdl or verilog in order to read in the VHDL or Verilog source RTL files.
- $HDL_NETL is assigned to ‘hdl_out’ to define the output format for netlist and SDF files. It can be set to either vhdl or verilog to generate either VHDL or Verilog netlists.

If the environment variables are not required, then the scripts can be modified to directly assign values.

4.6.3 Constraint Scripts

a) Operating conditions and signal load/drive values should be set for the chosen technology library in a file which is equivalent to the ‘library_avanti_cb25_v2.1.scr’ file.

b) Timing constraints for AHB bus signals are set via the amba_params.scr, ahb_master.scr and the ahb_slave.scr files. As delivered, the timings are set for a 100 MHz clock with 50% duty cycle. These timings should be adjusted according to the target frequency required. The above-mentioned files have been written using parameters and only requires amendment of the clock frequency with all other bus parameters (except clock skew) scaling accordingly.

   Clock skew needs to be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values have been used as preliminary values prior to layout:
   - the minus uncertainty value is set at 5% of the clock period.
   - the plus uncertainty value is set at 40% of the minus uncertainty value.
   The plus uncertainty derives from best case condition and it is less than the minus uncertainty value which derives from worst case condition.

   The timing parameters assume the following bus timing budgets :

   - 30 % input setup on all input ports including those on the AHB Slave ports and the AHB Master ports.
   - 40 % output valid delay on all outputs including those on the AHB Slave ports and the AHB Master ports.

c) Timing constraints for non-AMBA SMC-specific signals are set via the Smc.scr file. Similarly for the lower level SmcCore, DBI and TIC blocks, the timing constraints for non-AMBA signals are set in the corresponding SmcCore.scr, DBI.scr, DBI_tic.scr and TIC.scr files respectively. Sample input and output delay constraints are specified for these signals. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.

d) Timing exceptions files are used to specify false paths. The exception file corresponding to each of the block with some explanations are enumerated as follows :

- Top level SMC : the exceptions are defined in the Smc_exceptions.scr file. False paths are specified for the TIC block I/O paths since it is not expected to run at full speed (100 MHz). False paths are also specified for the tied input pins going to the SmcCore and DBI blocks.
- For the SmcCore block, the exceptions are set in the SmcCore_exceptions.scr file, which specifies false paths for the tied input pins. This file is called by the SmcCore.cmd compile script when synthesising the SmcCore block.
- In case of DBI block, two exception files have been defined. The DBI_exceptions.scr file contains the false path definition for tied input pin. The second file, DBI_tic_exceptions.scr, is used for setting false paths on all the SmcCore related paths when the TIC is active. These files are included by the DBI.cmd compile script when synthesising the DBI block.

e) A common script file, called Smc_Busmode.scr, is used to set value on the **EXTBUSMUX** input port to choose between the DBI block in SMC or an external EBI, as discussed earlier. If the **EXTBUSMUX** pin tie off value is logic ‘1’, then during synthesis the compiler can optimise out the redundant DBI block. This script is included by the master compile command scripts Smc.cmd and DBI.cmd.

Note The synthesis scripts assume input clocks (**HCLK** and **nHCLK**) have a 50% duty cycle. If clocks are used which do not have a 50% duty cycle then the constraints of the falling edge clocked outputs (**SMnWEN** and **SMnBLS**) must be adjusted appropriately.

4.7 Integration with AMBA Micropack components

The Static Memory Controller peripheral is designed to be integrated into an AMBA system via an AHB (Advanced High Performance Bus) slave port and an AHB Master port. The AHB interface through the AHB slave port provides access to the SmcCore block. The other block in the SMC is the TIC, which will be used for system testing through its AHB Master ports.

The low gate count TIC block allows externally applied test vectors to be converted into internal AMBA bus transfers. The AMBA test philosophy allows individual modules in the system to be tested in isolation by only using transfers from the bus. It does not rely on interaction of other system elements such as a processor core.

For more information on system test, refer to the AMBA Specification [Ref.1] and the Example AMBA System Technical Reference Manual [Ref.4]. During system test, HCLK is used as TCLK and SMDATAIN/SMDATAOUT as TBUSIN/TBUSOUT. The TIC signal TicRead controls the external bus interface drivers through the nSMDATAEN[3:0] output pads.

The TIC shares the external bus along with the SmcCore with the help of the DBI block, which is responsible for the bus arbitration.

Note If used, the TIC should be connected as the highest priority master on the AHB, but must not be connected as the default master. In a standard system, the CPU is connected as the default master.

4.8 Clocking

The ARM PrimeCell SMC contains two clock sources at the top level – HCLK and nHCLK. Two scan chains are formed, one for each clock domain.

4.9 Clock Tree synthesis

Many different design methodologies are available for clock tree insertion and synthesis. The default Static Memory Controller synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that clock buffering will be performed by a separate tool, such as place & route tools which understand placement, further on in the backend process. It is possible to modify the scripts if this is not the preferred approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during the place and route stage and not during synthesis.

4.10 Resets

The SMC block has a single reset pin. The system bus reset signal HRESETn, resets all internal registers. The HRESETn can be asserted asynchronously by the system reset controller but it should be deasserted synchronous to the rising edge of HCLK.

4.11 Reset Buffering

One of several available methods should be used to ensure correct buffering of the reset nets. The reset signals could be assigned infinite drive strength allowing a chip-level integration process to ensure balanced reset trees are used at the top-level of the chip or propagating the tree down to all end points. Another alternative is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block.

4.12 Synchronisers

The asynchronous SMC input signals are synchronised by a series of two flip-flops inferred during synthesis from standard HDL templates. The two asynchronous input pins SMWAIT and CANCELSMWAIT are used in the SmcCore block.

Some technology libraries offer meta-stability hardened D flip-flops specifically for implementing synchronisers. All the synchronisers in the SmcCore block have been isolated to the SmcSynchroniser sub-block inside the SmcCore. This allows the synthesis scripts to be modified so that these blocks could be synthesised separately to use the synchroniser flip-flop cells using the Synopsys Design Compiler **set_prefer** command. Further synthesis of these sub-blocks can then be prevented using the **set_dont_touch** command. The rest of the SMC design is then synthesised after placing a **set_dont_use** command on the synchroniser cells. This method also offers the opportunity to simulate the gate-level netlist with asynchronous SMC inputs and eliminate the expected setup and hold violations on the synchronisers from the simulation transcript, since the synchronisers cells can be modified to remove ‘X’ propagation.

4.13 Production Testing

The Static Memory Controller is designed to be production tested using scan-insertion and ATPG.

4.13.1 Scan Insertion and ATPG

It is assumed that the user will already be familiar with the design flow for scan insertion and ATPG, so only a brief discussion of integration issues which can arise are given below.

The Static Memory Controller design fully supports this scan test and ATPG methodology. Depending on the clocking arrangement used, the number of chip level scan chains required may vary. As delivered, the Static Memory Controller is synthesised with 2 scan-chains.

If scan insertion is to be performed post-synthesis, it should be done once the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or through use of equivalence checking. Scan insertion is then performed and the logical equivalence again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach may be adopted for synthesis where the RTL is compiled to create netlist with a scan test D flips-flops and provisionally connected scan chains. After scan insertion logical equivalence should again be proven.

The main advantage of the scan insertion and ATPG approach is in cases where significant modifications have been made to the Static Memory Controller block design by the user, or when most other user-generated parts of the chip use scan testing. Another advantage is that, a very high fault coverage can be achieved (99%).

Extraction of test vectors is done by the system designer once the whole chip design has been integrated. The quoted fault coverage (excepting variation resulting from the use of a sparse cell library) will be met by extracting test vectors around the complete chip while running the automatically generated vectors (ATPG).

4.14 Functional and Timing Verification

It is essential to verify timing performance of the netlist against the RTL at key stages in the design flow, post-synthesis, post-scan insertion, during layout and at completion before sign-off of the ASIC.

Gate-level simulation can be used for limited timing verification but it is slow and cannot exercise all timing paths in the design. Results tend to be optimistic since worst case timing paths may not be exercised.

Static timing verification is a worthwhile and efficient method of verifying timing performance of a design block, but can tend to be pessimistic unless false paths are defined to exclude such paths from analysis. Use of gate-level

simulation as well as Static Timing Analysis is a useful check for human error in the manual definition of false paths.

The netlist should also be checked for functional equivalence to the RTL at the same key stages of the design flow.

This can be performed by dynamic simulation using test vectors as described above, or by use of a logic equivalence checking tool (For example, Synopsys Formality or Chrysalis).

5 APPENDIX A

5.1 PrimeCell Environment Variables

Simulation and Synthesis in the Static Memory Controller world are controlled by a set of environment variables.

GLOBAL

The path to the shared files is defined by the **'GLOBAL'** variable.

e.g. : **setenv GLOBAL /h/arm/global**

PERIPH

The peripheral name is defined by the **'PERIPH'** variable.

e.g.: **setenv PERIPH Smc**

SIMULATOR

The simulator type is defined by the **'SIMULATOR'** variable.

It can take the following two values

- modelsim : for ModelSim simulations
- LDV : for LDV simulations

e.g. : **setenv SIMULATOR modelsim**

setenv SIMULATOR LDV

TEST_METH

The scan methodology adopted during synthesis is defined by the **'TEST_METH'** variable.

It can take the following three values

- noscan : creates a netlist without scan structures
- scanready : creates a netlist with scan F/Fs but without scan routing.
- scaninsert : creates a fully routed scan netlist.

e.g. : **setenv TEST_METH noscan**

setenv TEST_METH scanready

setenv TEST_METH scaninsert

HDL_SOURCE

The source code type is defined by the **'HDL_SOURCE'** variable.

It can take the following two values

- vhdl [VHDL RTL]
- verilog [Verilog RTL]

e.g. : **setenv HDL_SOURCE vhdl**

setenv HDL_SOURCE verilog

HDL_NETL

The netlist type is defined by the **'HDL_NETL**' variable.

It can take the following two values

- vhdl [VHDL netlist]
- verilog [Verilog netlist]

e.g. : **setenv HDL_NETL vhdl**

setenv HDL_NETL verilog

TEST_ENV

The test environment is defined by the **'TEST_ENV**' variable.

It can take the following two values

- BUSTALK [For functional vector simulations]
- TICTALK [For integration vector simulations]

e.g. : **setenv TEST_ENV BUSTALK**

setenv TEST_ENV TICTALK

LIB_VHDL

The location of the vhdl cell library views is defined by the **'LIB_VHDL**' variable.

e.g. : **setenv LIB_VHDL /AVANT/1999.1/cb25/v2.1/vital/cb25os163/zero**

DIRS_VERILOG

The location of the verilog cell library views is defined by the **'DIRS_VERILOG'** variable.

e.g. : **setenv DIRS_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero**

FILES_VERILOG

The location of the verilog **UDP** files is defined by the **'FILES_VERILOG'** variable.

e.g. : **setenv FILES_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero/mtb_verilog.v**

LIB_SYNOP

The location of the synopsys target library files is defined by the **'LIB_SYNOP'** variable.

e.g.: **setenv LIB_SYNOP /AVANT/1999.1/cb25/v2.1/synopsys/cb25os163/1997.01/models**

AMBA

This environment variable defines the AMBA bus to which the peripheral is connected - AHB, ASB or APB. Since the SMC PL092 is an AHB peripheral, set this environment variable to 'AHB'. This environment variable is used by makefiles in the SMC database to select settings for the Integration world.

e.g. : **setenv AMBA AHB**